

AI Compiler Performance Engineering

Auto-Loops Books — ch02

Contents

1	Two Kernel Design Mindsets: Operator-Driven vs Dataflow-Driven	1
1.1	Operator First	1
1.2	Data Residency	2
1.3	Dataflow Philosophy	3
1.4	Cross Hardware	4
1.5	Iron Rules	5
1.6	Megakernel Preview	6
1.7	Automation Path	6
1.8	Contrasting Schedules on One Decoder Layer	7
1.9	Chapter Summary	10
	Bibliography	12

Chapter 1

Two Kernel Design Mindsets: Operator-Driven vs Dataflow-Driven

Chapter ?? showed *what* breaks in decode: launch storms, HBM round trips, and hardware-specific bottlenecks. A durable industry misconception is that the fix is “one more fused op” inside an otherwise operator-first stack—when the schedule itself still exports every intermediate to global memory between mathematically adjacent compute stages (Vellaisamy *et al.*, 2026; Chen, 2026). This chapter completes the pivot: from **operator-driven** schedules (compute-first, memory-second) to **dataflow-driven** schedules (residency-first, compute as a consequence of placement). Readers should leave with a decision procedure: given a decoder bottleneck measurement (kernels/token, bytes/token, HDBI, R_{floor}), classify whether the next engineering dollar belongs in op micro-tuning or dataflow replanning—and default to replanning when Chapter 1 diagnostics show orchestration or HBM export dominance (Vellaisamy *et al.*, 2026; Chen, 2026).

1.1 Operator First

In production, most CUDA teams still begin decoder optimization by listing operators: LayerNorm, QKV projection, attention, MLP, residual adds. Each op receives a tuned kernel; the runtime stitches them with separate launches. The failure mode is predictable: *compute* micro-optimizations improve individual kernels while end-to-end ms/token stalls because every boundary writes activations



Figure 1.1: Operator-driven schedule: each op reads/writes global memory.

to HBM and pays another launch (Vellaisamy *et al.*, 2026). Operator-first design treats memory as an implementation detail behind each op. PyTorch eager mode is the canonical embodiment: the graph is correct, but the schedule is emergent. FlashAttention grafts faster replacements for single nodes; it does not change export/import between nodes unless a compiler pass merges regions (Dao *et al.*, 2022). On GPUs this yields hundreds to thousands of kernels per decode token; on CPUs LLC churn; on NPUs illegal dynamic schedules (AMD, 2024b). Teams can spend quarters shaving nanoseconds inside MMA loops while HDBI remains near zero (Vellaisamy *et al.*, 2026; Chen, 2026). The operator-first workflow also fragments accountability: kernel owners optimize “their” op, while no team owns the HBM edges between ops—yet those edges dominate decode (Chen, 2026). Compiler stacks that expose only per-op autotuning (template GEMMs, library dispatch tables) reinforce this silo unless a higher pass owns cross-op lifetime analysis. Dataflow-first teams invert the org chart: one schedule owner, many op specializations *inside* fused regions.

Example 1.1.1. A dense 7B decoder layer on H100 at batch-1 may spend $\sim 15\text{--}25\mu\text{s}$ in device compute per op group but $\sim 30\text{--}50\mu\text{s}$ waiting on launches and framework translation between groups (Chen, 2026; Vellaisamy *et al.*, 2026). An operator-first team that speeds each compute group by 20% saves only $\sim 3\text{--}5\mu\text{s}$ aggregate while launch gaps remain. A dataflow-first team that fuses three groups into one dispatch may eliminate two launch gaps ($\sim 60\text{--}100\mu\text{s}$ class savings on launch-bound configs) even if per-op compute is unchanged. The example is qualitative—exact numbers vary by model and driver—but the *ranking* of leverage is what production teams must internalize.

1.2 Data Residency

The root cause is not insufficient FLOPs; it is **data residency** violated at every layer boundary. Activations that could live in registers or shared memory for the full QKV $\rightarrow$ attention $\rightarrow$ MLP chain are materialized to global memory after each sub-step. Each round trip pays full HBM bandwidth and pollutes CPU caches (Chen, 2026; Davies *et al.*, 2025). Residency is a compile-time allocation problem: which tensors coexist, which tier holds them, for how many fused stages? On

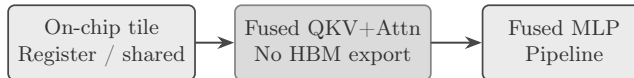


Figure 1.2: Dataflow-driven schedule: activations stay on-chip across fused stages.

GPUs the tiers are registers $\rightarrow$ shared $\rightarrow$ L2 $\rightarrow$ HBM; on CPUs cache and NUMA; on XDNA/AIE hardware-enforced tile buffers (NVIDIA, 2022; AMD, 2024b,a). When residency succeeds, arithmetic intensity rises because bytes moved drop even if FLOPs stay constant (Davies *et al.*, 2025). PagedAttention improves KV capacity but does not guarantee on-chip residency inside a fused decoder region (Kwon *et al.*, 2023). Quantization changes residency economics: int4 weights reduce bytes streamed per layer, but only when kernels truly consume compressed layouts without dequant scratch in HBM (Chen, 2026). A residency plan must therefore co-design precision, layout, and fusion: nf4 weights with bf16 activations still ping-pong if every op dequantizes to global buffers. Industrial teams should measure *bytes per token* and *unique HBM touch count* per layer alongside FLOPs—metrics that residency planning directly minimizes.

Residency analysis also exposes **anti-locality** in popular serving features. Continuous batching improves throughput by reusing weight matrices across requests, but if each request’s activations still round-trip through HBM inside the layer, tail latency per request remains decode-bound (Kwon *et al.*, 2023). Speculative decoding adds draft-model traffic—another residency consumer—that operator-first stacks often bolt on without revisiting fusion boundaries. Dataflow planners must account for multi-model residency simultaneously: base model weights, draft model weights, and KV streams competing for bandwidth and on-chip space (Davies *et al.*, 2025). The book’s later compiler passes treat these as first-class buffer interference constraints rather than afterthought flags.

1.3 Dataflow Philosophy

Dataflow design names the schedule before operators. Four commitments recur across targets:

1. **Static placement** where possible—tensor lifetimes planned before codegen; NPU require static ADF graphs (AMD, 2024b).
2. **On-chip residency first**—fusion depth keeps producer–consumer pairs in the fastest tier.

Table 1.1: Design mindset vs. hardware enforcement model.

Target	Data residency model
CUDA GPU	Software-managed hierarchy
XDNA / AIE	Hardware-enforced spatial buffers
CPU	Cache hierarchy + explicit blocking

3. **Pipelined movement**—TMA on Hopper and DMA on AIE hide transfers when the graph exposes prefetch edges (NVIDIA, 2022; AMD, 2024a).
4. **Minimal synchronization**—dispatch unit is the pipeline stage, not each elementary op.

Porting a CUDA operator list to XDNA without a static pipeline yields no legal schedule. On GPUs, ignoring dataflow still “works”; on NPUs compile fails—explaining why operator-first CUDA expertise does not automatically transfer to edge fleets. Static dataflow does not mean frozen models: it means shapes and buffer slots are known at compile boundaries (per deployment SKU), with guarded recompilation when service configs change. Serving systems that mix dynamic batching with static fusion must separate *scheduling* concerns (which requests share a batch) from *kernel* concerns (what stays on-chip during a fused step) (Kwon *et al.*, 2023). Confusing the two produces graphs that are dynamically batched at the framework level but still operator-fragmented inside each batch member—the worst of both worlds for tail latency.

Dataflow philosophy also reframes **correctness work**. Operator-first testing validates per-op numerical parity against reference implementations. Dataflow-first testing must add **schedule parity**: fused pipelines must match unfused references within tolerance while preserving tier assignments across backends. Regression suites therefore include golden activation dumps at stage boundaries during development, then strip those dumps in production builds once parity is proven—a pattern YiRage CI adopts for multi-target codegen. The extra testing cost pays back when a single fusion bug would otherwise ship as a $1.5\times$ silent slowdown on only one hardware class.

1.4 Cross Hardware

Cross-hardware comparison is not an appendix exercise—it is how teams avoid shipping GPU-tuned schedules that silently fail on edge NPUs or underperform

on CPU inference nodes co-located with GPU fleets (AMD, 2024b; Chen, 2026). The book’s benchmark harness (Appendix B) always reports at least three SKUs when hardware access allows: one bandwidth-rich GPU, one orchestration-sensitive GPU, and one CPU or NPU class target. Mindset differences show up as *different optimal fusion depths* on the same mathematical model, not as different models.

For **CUDA GPUs**, shared memory sizing, register pressure, and occupancy set fusion depth. CUDA Graphs amortize launches but replay a fixed dataflow; they do not invent residency that eager boundaries destroyed (NVIDIA, 2024; Chen, 2026). Hopper TMA raises async movement ceilings but needs static copy edges (NVIDIA, 2022; SemiAnalysis, 2024). For **CPUs**, residency maps to **cache** blocking and NUMA placement; GPU-style mega-fusion can evict KV working sets. SIMD kernels benefit when dataflow keeps head layouts hot across QKV and MLP epilogues. For **XDNA / AIE**, residency is hardware-enforced: spatial tiles, DMA routes, compile-time buffer tables (AMD, 2024b,a). Dynamic shapes and per-op dispatch are mismatches; the dataflow graph *is* the program. One mindset expresses differently per target, but operator-first lists fail everywhere for the same reason—no global residency contract. Table 1.1 summarizes enforcement models; the engineering consequence is that **fusion granularity** differs by class: GPUs tolerate medium fusion with shared-memory staging; CPUs need cache-aware blocking; NPUs require layer-scale or multi-layer static regions (AMD, 2024b). Cross-hardware portability is portability of the *dataflow graph and lifetime annotations*, not of individual op implementations. YiRage backends swap codegen while preserving tier assignments—the topic of Parts IV–VI.

1.5 Iron Rules

Rule 1 — Registers hold ephemeral scalars and narrow vectors, not whole feature maps; spills mean the split is wrong (NVIDIA, 2022). **Rule 2 — Shared memory / LDS / on-chip SRAM** holds fusion regions; depth ends at capacity conflicts, not when the op list ends. **Rule 3 — Global memory** is for weights, KV streams, and true outputs—not ping-pong activations between adjacent ops (Kwon *et al.*, 2023; Chen, 2026). Violations produce Chapter 1 pathologies: high kernels/token and low R_{floor} when export/import dominates (Vellaisamy *et al.*, 2026; Chen, 2026). Use iron rules as release gates: if a proposed schedule inserts a global round trip between ops that share a producer–consumer edge in the mathematical graph, reject it unless profiling proves the tier overflow is unavoidable. On GPUs, “unavoidable” should trigger shared-memory tiling or pipeline splitting, not acceptance of HBM ping-pong. On NPUs, overflow should fail compile and force graph refactoring—not

runtime fallback loops on the host.

Register pressure on GPUs frequently triggers the false comfort of “spilling is fine because the kernel still runs.” In decode, spilled activations often become HBM traffic that negates fusion (NVIDIA, 2022; Chen, 2026). Iron Rule 1 therefore pairs with occupancy discipline: if widening tile sizes forces spills, shrink fusion depth instead of celebrating higher theoretical occupancy on paper. CPU analogs apply to stack spills and false sharing when fusing across threads without cache-line-aware partitioning.

1.6 Megakernel Preview

A **MegaKernel** dispatches an entire decoder layer with activations resident across QKV, attention, MLP, norms, and residuals—the antithesis of Figure 1.1. MegaKernels are fusion schedules with staged pipelines, not one giant matmul (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Chaining vendor libraries without shared residency drops launches modestly but often not HBM traffic. True MegaKernels collapse both; TaxBreak still shows order-of-magnitude launch gaps on MoE graphs after partial fusion (Vellaisamy *et al.*, 2026). They trade flexibility for performance—acceptable when model revisions are pinned per fleet. Part V develops manual craft; Part VI automates the same decisions in YiRage. MegaKernel engineering also clarifies **where not to fuse**: attention kernels that require global KV streaming may remain distinct stages while MLP+norm+residual fuse around them—dataflow does not mandate monolithic single-kernel binaries, it mandates explicit staging with minimal exports. Flash-Decoding parallelizes KV dimension while keeping decode batch-1 viable (Dao *et al.*, 2023); a MegaKernel schedule must *compose* with such attention stages rather than naively inlining them and destroying register budgets.

1.7 Automation Path

YiRage ingests decoder graphs, plans lifetimes globally, runs fusion passes, and emits CUDA, CPU, or XDNA backends from one dataflow IR (AMD, 2024b; NVIDIA, 2022). The automation path encodes expert workflow—probe hardware, plan residency, fuse, codegen, regress—as compiler logic. Each backend re-instantiates the same dataflow with different tier rules: GPU shared-memory fusion and graph capture (NVIDIA, 2024); CPU cache blocking; XDNA static buffers and DMA ping-pong (AMD, 2024a). YiRage fails compile when residency

cannot be proved—preferable to silent production slowdowns. Optimize dataflow first; let operators fall out of the schedule second. Automation must preserve human-debuggable artifacts: lifetime graphs, fusion decision logs, and regression tables comparing bytes/token and kernels/token against PyTorch eager and framework servers (Vellaisamy *et al.*, 2026). Without those artifacts, compiler automation reintroduces opacity—teams cannot tell whether a regression is a bad tile size or a violated iron rule. YiRage treats diagnostics as first-class outputs alongside generated kernels, echoing TaxBreak and memory-floor methodologies from Chapter 1.

1.7.1 Compiler IR View

At the compiler layer, the mindset shift appears as a change in primary IR artifacts. Operator-first pipelines lower high-level frameworks (Torch, ONNX) into op sequences early; each op lowers independently to LLVM, Triton, or CUDA C++ with late, local fusion peephole passes. Dataflow-first pipelines retain a *lifetime graph* on tensors until hardware tier assignment completes: SSA values carry storage class annotations (`reg`, `shared`, `global_stream`) and fusion legality is checked on inter-value edges. GPU backends emit kernels per fused region; CPU backends emit blocked loops; XDNA backends emit ADF graphs with DMA edges (AMD, 2024b). The IR difference is why “drop in FlashAttention” is not equivalent to “run the fusion pass pipeline”: the former swaps an op implementation; the latter rewrites the graph topology.

1.7.2 Failure Modes When Mixing Mindsets

Hybrid stacks—dataflow planning in a planner service but operator-first codegen in backends—produce characteristic failures. **Planner drift:** fusion regions computed on a meta-graph diverge from what the backend actually emits, reintroducing hidden exports. **Partial graph capture:** CUDA Graphs record a subgraph that still contains HBM ping-pong inside recorded nodes (NVIDIA, 2024). **False residency:** shared-memory buffers declared fused but spilled registers silently increase global traffic (NVIDIA, 2022). **NPU illegalize late:** graphs accepted on GPU fail XDNA buffer allocation in production edge builds (AMD, 2024a). Guard against hybrids with single-source dataflow IR and fail-fast legality checks per backend, the YiRage default posture.

1.8 Contrasting Schedules on One Decoder Layer

Consider a single transformer decoder layer at batch-1 as the book’s running microscope. **Operator-first execution** issues separate kernels for RMSNorm, QKV linear, rotary embedding, attention, output projection, second RMSNorm, gate/up/down MLP projections, activation, and residual adds—each reading inputs from HBM and writing outputs back. Even if every kernel is “optimized,” the layer may execute ~ 10 – 20 global export/import cycles per token per layer, multiplied by host launches documented in TaxBreak (Vellaisamy *et al.*, 2026). Profiler heatmaps look healthy inside each kernel; end-to-end latency does not move.

Dataflow-first execution partitions the same mathematics into two or three pipeline stages with explicit buffer assignments. Stage A might fuse RMSNorm+QKV+rotary into shared-memory tiles; Stage B runs attention with KV streaming while keeping query tensors resident; Stage C fuses MLP+norm+residual without global activation ping-pong. Launches drop from ~ 10 – 20 toward 1–3; bytes moved drop because producer–consumer pairs never left the fastest tier. On H100-class GPUs this is how CUDA Graphs and MegaKernels compound: graphs fix dispatch; dataflow fixes bytes (NVIDIA, 2024; Chen, 2026).

The contrast is sharper on **CPU**: operator-first eager PyTorch thrashes LLC moving the same activation lines through DRAM for every sub-op. Dataflow-first blocking keeps the MLP epilogue in L2 while weights stream once per layer. On **XDNA**, operator-first schedules may not legalize at all; dataflow-first ADF graphs assign each stage to tile columns with DMA prefetch of weights while SRAM holds activations (AMD, 2024b,a). The layer microscope therefore generalizes: the mathematical layer is fixed; the *schedule topology* is the optimization variable.

1.8.1 Migration Checklist: Operator Stack to Dataflow IR

Teams porting production decoder stacks should not “rewrite everything” day one. A pragmatic migration applies five checks to each layer group:

1. **Export audit.** List tensors written to global memory between mathematically fused-able ops; each edge is a candidate fusion edge.
2. **Launch audit.** Count kernels per token; if $\gg 100$ on dense decode, graph capture or fusion passes are higher ROI than MMA tuning (Vellaisamy *et al.*, 2026).
3. **Tier budget.** Estimate shared-memory or cache footprint for candidate fusion; split stages when capacity exceeds hardware limits rather than spilling

silently.

4. **Attention boundary.** Keep KV-global attention stages explicit unless register/shared budgets prove full inlining safe (Dao *et al.*, 2023, 2022).
5. **Regression matrix.** Re-run bytes/token and ms/token on GPU, CPU, and NPU SKUs after each fusion tranche—not only the device that authored the kernel.

YiRage encodes these checks as pass gates; manual teams should encode them as CI benchmarks even before automation lands.

1.8.2 Organizational Incentives

Operator-first stacks persist partly because performance reviews reward visible kernel wins—ns saved in cuBLAS, occupancy graphs in Nsight—while dataflow wins appear “distributed” across fewer exports and launches. Staffing mirrors the stack: siloed kernel engineers without a compiler owner cannot hold global residency accountability. Industrial leaders align incentives with bytes/token and kernels/token OKRs tied to product p99 latency, not single-kernel FLOP counters (Chen, 2026; Vellaisamy *et al.*, 2026). This organizational point matters for adoption: dataflow-first design is a *process* change as much as a code change.

1.8.3 Glossary Bridge

To align vocabulary across hardware chapters: **operator** means a mathematically named function with its own launch or library entry; **stage** means a fused pipeline region with one dispatch and internal register/shared residency; **dataflow** means the directed acyclic graph of tensor lifetimes and movement edges; **residency** means the storage tier holding a value at a program point; **MegaKernel** means a manually authored stage at or near full-layer scope; **automation path** means compiler passes that derive stages from IR instead of hand CUDA/HIP. These terms are used consistently in Chapters 3–11 and in YiRage pass names in Part VI.

1.8.4 Historical Note: Why Operator-First Won (Temporarily)

Operator-first CUDA culture dominated the 2010s because training workloads were compute-heavy, batches were large, and framework ecosystems competed on op coverage. Inference—especially decode—was treated as “training with batch 1” long after production traffic proved otherwise (Davies *et al.*, 2025).

FlashAttention, PagedAttention, and graph capture each corrected a slice of the inference gap without overturning the op-list mental model (Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024). The dataflow-first resurgence in this book is driven by deployment reality: disaggregated decode fleets, MoE launch inflation, edge NPUs, and cross-hardware SKUs that cannot afford per-device hero kernels (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024b). Understanding *why* operator-first habits persist helps teams plan migrations without dismissing past kernel investments—those kernels become *epilogues inside stages*, not the architecture itself.

1.8.5 Practice Problems (Engineering, Not Exercises)

Use these audits on your own decoder deployment:

1. Draw the operator graph for one layer and mark every HBM read/write edge; count edges per token.
2. Propose a two-stage dataflow that removes at least half those edges; estimate shared-memory or cache bytes required.
3. Run TaxBreak-style launch counting or equivalent framework profilers; compare kernels/token before and after fusion (Vellaisamy *et al.*, 2026).
4. Measure R_{floor} on your GPU SKU at batch-1; if high, prioritize bytes; if low, prioritize launches (Chen, 2026).
5. If targeting NPU, attempt to legalize the same graph statically; document which edges force DDR spill (AMD, 2024b).

Solutions appear implicitly in Parts V–VI through YiRage pass implementations; the audits themselves are portable across vendors.

Readers building internal playbooks should copy the audits into onboarding docs for new kernel hires—especially those arriving from training-focused CUDA backgrounds—so decode residency constraints are explicit on day one rather than rediscovered via quarter-long latency firefights (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The audits also give managers a neutral vocabulary when negotiating roadmap priority between framework teams (batching, KV paging) and compiler teams (fusion passes): both sides optimize different layers of the same dataflow, and the checklists show where hand-offs must be contractually precise (Kwon *et al.*, 2023).

1.9 Chapter Summary

Operator-driven design optimizes kernels in isolation; dataflow-driven design optimizes *where data lives* across fused stages. Decode on GPUs, CPUs, and NPUs converges on residency and launch granularity (Davies *et al.*, 2025; Chen, 2026). Iron rules on register, shared, and global tiers make philosophy checkable. MegaKernels preview the manual ceiling; YiRage previews industrial multi-hardware scale. Chapter 3 grounds these abstractions in architecture limits every later pass must model. The through-line for the reader is simple: stop asking “which op is slow?” in isolation and start asking “which residency contract did we violate, on which hardware, and how does the compiler re-plan the layer?”—that question scales from manual MegaKernels to automated YiRage deployments without changing the engineering logic (Vellaisamy *et al.*, 2026; Chen, 2026; AMD, 2024b). Carry this mindset into every hardware chapter that follows: silicon details change, but the residency contract does not. When Chapter 3 maps bandwidth floors and SRAM caps, read them as *inputs* to the dataflow planner—not as footnotes to individual operator tiles. When Part V shows handwritten CUDA MegaKernels, read them as *one legal schedule* that YiRage will generalize—not as a gallery of unrelated tricks. Finally, treat every performance regression in later chapters as a residency or dispatch regression until proven otherwise—that default suspicion saves weeks of misdirected op tuning and keeps the book’s engineering narrative coherent across GPU, CPU, and NPU fleets (Vellaisamy *et al.*, 2026; Chen, 2026). If this chapter changed only one habit, let it be the habit of drawing the dataflow before opening the profiler on a single kernel—the drawing is cheaper, and it usually names the real bottleneck correctly the first time.

Bibliography

- AMD (2024a). Ai engine technology. <https://www.amd.com/en/products/adaptive-socs-and-fpgas/technologies/ai-engine.html>.
- AMD (2024b). AMD XDNA architecture. <https://www.amd.com/en/technologies/xdna.html>.
- Chen, J. (2026). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4.
- Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8× decode speedup at long context.
- Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*.
- Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefill nodes.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*.
- Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.
- NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog.
- NVIDIA (2024). Cuda graphs. CUDA Programming Guide.
- SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter.

Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*.